

Lab9 Report

22300240022 王镜凯

代码功能分析报告：汇编代码生成器实现

1. 总体功能概述

这段代码是一个中间代码到汇编代码的转换函数 `convert`，主要功能是将四元式（Quad）表示的中间代码转换为 ARM 汇编代码。该函数处理了函数定义、指令生成、寄存器分配与溢出处理、指令优化等多个方面，最终生成可执行的汇编代码片段。

2. 关键数据结构与参数说明

2.1 主要输入参数

- `QuadFuncDecl* func`：四元式函数声明，包含函数名和四元式块列表
- `DataFlowInfo *dfi`：数据流信息，可能用于寄存器分配和优化
- `Color *color`：寄存器分配结果，记录临时变量到寄存器的映射及溢出信息
- `int indent`：缩进级别，用于生成格式化的汇编代码

2.2 核心数据结构

- 四元式（Quad）：表示中间代码的基本单元，包含多种类型（MOVE、JUMP、CJUMP 等）
- 临时变量（Temp）：中间代码中的临时存储单元，可能映射到寄存器或栈
- 标签（Label）：用于跳转指令的目标标识

3. 代码执行流程分析

3.1 函数头部生成

```
result += "\n.balign 4\n";
result += ".global " + normalizeName(func->funcname) + "\n";
result += ".section .text\n\n";
result += normalizeName(func->funcname) + ":\n";
```

- 生成汇编文件的头部信息，包括内存对齐、全局函数声明和代码段定义

3.2 栈帧初始化

```
result += string(indent, ' ') + "push {r4-r10, fp, lr}\n";
result += string(indent, ' ') + "add fp, sp, #32\n";
```

- 保存非易失性寄存器和链接寄存器

- 初始化帧指针 (fp) , 指向栈顶 + 32 字节位置

3.3 溢出空间分配

```
int spill_size = color->spills.size() * 4;
if (spill_size > 0) {
    result += string(indent, ' ') + "sub sp, sp, #" + to_string(spill_size) + "\n";
}
```

- 为溢出到栈的临时变量分配空间 (每个变量 4 字节)
- 更新栈指针以预留溢出空间

4. 四元式到汇编的转换逻辑

4.1 标签 (LABEL) 处理

```
if (stm->kind == QuadKind::LABEL) {
    // 检查并删除多余的跳转指令
    result += convert(label, color, indent);
}
```

- 生成标签定义, 并优化相邻的冗余跳转指令

4.2 数据移动 (MOVE) 指令

```
case QuadKind::MOVE: {
    // 处理源和目标操作数的寄存器映射
    if (src == dst) break; // 跳过冗余移动
    result += "mov " + dst + ", " + src + "\n";
}
```

- 将临时变量或常量从源位置移动到目标位置
- 处理寄存器溢出情况, 从栈加载或存储到栈

4.3 二元运算 (MOVE_BINOP) 指令

```
case QuadKind::MOVE_BINOP: {
    // 支持+、-、*、/等运算
    if (binop->binop == "+") result += "add " + dst + "...";
    // 指令合并优化: 将add与后续的LOAD/STORE合并
}
```

- 生成算术运算的汇编指令
- 实现指令优化: 当遇到 `add x, y, #n` 后接 LOAD/STORE 时, 合并为地址计算

4.4 跳转 (JUMP) 与条件跳转 (CJUMP) 指令

```

case QuadKind::JUMP: {
    result += "b " + label->str() + "\n";
}
case QuadKind::CJUMP: {
    result += "cmp " + src1 + ", " + src2 + "\n";
    // 根据比较结果生成beq、bne等条件跳转
}

```

- 生成无条件跳转和条件跳转指令
- 使用 `cmp` 指令进行比较，后跟条件跳转指令

4.5 函数调用 (CALL) 与返回 (RETURN)

```

case QuadKind::CALL: {
    // 前4个参数存入r0-r3寄存器
    result += "blx " + obj_ptr + "\n"; // 带链接的跳转
}
case QuadKind::RETURN: {
    result += "mov r0, " + ret_reg + "\n"; // 设置返回值
    result += "pop {r4-r10, fp, pc}\n"; // 恢复寄存器并返回
}

```

- 处理函数调用，参数传递遵循 ARM AAPCS 规范
- 函数返回时恢复栈帧并将返回值存入 r0

4.6 内存加载 (LOAD) 与存储 (STORE)

```

case QuadKind::LOAD: {
    result += "ldr " + dst + ", [" + src + "]\n"; // 从内存加载到寄存器
}
case QuadKind::STORE: {
    result += "str " + src + ", [" + dst + "]\n"; // 从寄存器存储到内存
}

```

- 处理内存访问操作，支持变量和常量地址

4.7 Phi 节点处理

```

case QuadKind::PHI: {
    // 为每个分支生成赋值指令
    if (目标是溢出变量) result += "str ... \n";
}

```

- 处理控制流汇合点的变量合并操作
- 为每个前驱分支生成对应的赋值语句

5. 寄存器分配与溢出处理机制

5.1 溢出检测与处理

```
if (color->spills.find(temp->num) != color->spills.end()) {  
    result += "ldr r9, [fp, #-offset]\n"; // 从栈加载溢出变量  
    // 或  
    result += "str r10, [fp, #-offset]\n"; // 存储到栈  
}
```

- 通过 `Color` 对象的 `spills` 集合判断临时变量是否溢出
- 溢出变量通过栈指针 (`fp`) 的偏移量访问

5.2 寄存器重用策略

- 使用 `r9`、`r10` 作为临时寄存器处理溢出变量
- 对于未溢出变量，直接使用分配的寄存器（如 `r0-r3` 用于参数和返回值）

6. 指令优化技术

6.1 冗余指令消除

```
if (dst == src) break; // 跳过源和目标相同的移动指令
```

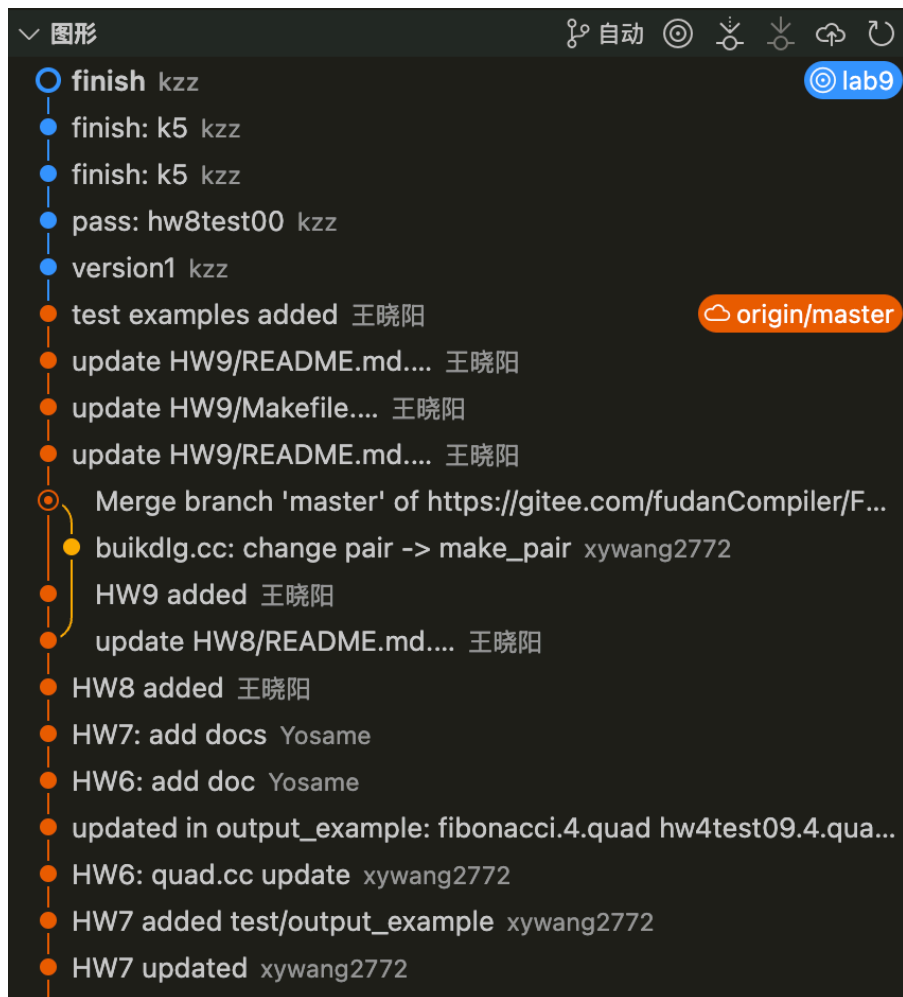
- 检测并跳过 `mov x, x` 之类的冗余指令

6.2 指令合并优化

```
if (binop->binop == "+" && nextStm是LOAD) {  
    // 合并为地址计算+加载  
    result += "ldr ... [src1, #const]\n";  
    skip = true; // 跳过下一条指令  
}
```

- 将 `add x, y, #n` 与后续的 `LOAD/STORE` 合并为单条指令
- 通过 `skip` 标志避免生成冗余指令

7. Git开发图



8. 参考资料

- PPT, 教材